AI600: Assignment 2 Report

Nifra Wahaj — 25280002

March 3, 2026

GitHub Repository: https://github.com/NifraWahaj/25280002_AI600_Assignment2

1 Dataset and Preprocessing

The dataset consists of 28×28 bitmap images belonging to 15 classes: *apple, baseball-bat, basketball, clock, compass, cookie, donut, ladder, mountain, pizza, rabbit, soccerball, spider, t-shirt,* and *wheel*. Three preprocessing steps were applied to all models:

1. **Flattening.** Each 28×28 image is reshaped into a vector of size 784 using `x.view(-1, 784)`. All spatial structure is discarded at this stage.
2. **Normalisation.** Pixel values are scaled from $[0, 255]$ to $[0, 1]$.
3. **Train/Validation Split.** 80% of samples are used for training and the remaining form a fixed validation set.

On-the-fly Data Augmentation (Champion only, applied in the training loop only – never to validation or test data):

1. **Horizontal flip** (50% probability per image) – doodle classes are largely mirror-symmetric.
2. **Random rectangular erase** (40% probability) – zeros out a random 3–10 × 3–10 pixel patch, simulating the incomplete doodles characteristic of Quick, Draw! data.
3. **Gaussian noise** (always applied, scale = 0.02) – adds small pixel-level perturbations to prevent memorisation of exact pixel values.

2 Part A: The Pancake

2.1 Architecture

The Pancake model is a wide, shallow network with 2 hidden layers.No BatchNorm is used and design is intentionally simple to isolate the effect of width.

$$784 \xrightarrow{\text{Linear}} 1024 \xrightarrow{\text{ReLU, Dropout(0.3)}} 512 \xrightarrow{\text{ReLU, Dropout(0.3)}} 15$$

Table 1: Pancake Model Architecture

Layer	**In**	**Out**	**After**
Input (flattened)	–	784	–
FC 1	784	1024	ReLU, Dropout(0.3)
FC 2	1024	512	ReLU, Dropout(0.3)
Output	512	15	raw logits

Parameters: 1,336,335
Epochs: 20
Optimiser: AdamW, $\eta = 10^{-3}$, weight decay 10^{-4}, cosine LR
Augmentation: Off

2.2 Results

Table 2: Pancake Training and Validation Performance

Metric	**Train (Epoch 20)**	**Val (Best)**
Accuracy	95.69%	78.54%
Loss	0.5011	0.9550

2.3 Analysis

The hypothesis was that raw width (packing many neurons into just two layers) would give the model enough capacity to solve classification. It didn't work out that way. Training accuracy climbed to 95.54% while validation accuracy plateaued around 78% from epoch 9 onward, leaving a train/val gap of roughly **17 percent**. The model was memorising training data rather than learning anything generalisable. This makes sense as width gives capacity but no structure. The model treats every pixel as equally related to every other and it has no idea that adjacent pixels are more meaningful than distant ones. So instead of learning that a pizza is a circle with radial slices, it learns correlations between specific pixel values that happen to hold in the training set but break on anything slightly different.

The Universal Approximation Theorem says a wide enough network *can* approximate any function, but it says nothing about whether gradient descent will actually find that solution or whether it will generalise. This experiment shows exactly that gap between what is theoretically possible and what happens in practice.

3 Part B: The Tower

3.1 Architecture

Table 3: Tower Model Architecture

Block	Output Dim	Components
Input	784	–
Blocks 1–7	256	Linear → BatchNorm → GELU → Dropout(0.2)
Output	15	Linear (raw logits)

Parameters: 603,151
Epochs: 20
Optimiser: AdamW, $\eta = 5 \times 10^{-4}$, weight decay 10^{-4}, cosine LR
Augmentation: Off

Deep networks are prone to the **vanishing gradient** problem so two mitigations were applied (without residual skip connections):

- **Batch Normalisation** where I normalised activations to zero mean and applied unit variance before the activation function to keep gradients well conditioned across depth.
- **GELU activation** which, unlike ReLU, has a smooth non zero gradient everywhere so there is less risk of dead neurons accumulating in a deep stack.

3.2 Results

Table 4: Tower Model Performance

Metric	Training (Epoch 20)	Validation (Best)
Accuracy	85.13%	78.66%
Loss	0.7291	0.9162

3.3 Analysis

The hypothesis was that depth would allow the network to learn hierarchical features using fewer parameters than the wide pancake model. This largely held up. The Tower's train/val gap is roughly **6%**, less than half the Pancake's 17 point gap and it achieves a higher best validation accuracy (78.66% vs 78.23%) with 733K fewer parameters. So depth with regularisation clearly generalises better than raw width.

Training was noticeably slower to get going. Epoch 1 only reached 43.74% training accuracy compared to 54.94% for the Pancake. This is consistent with the difficulty of training deep networks from random initialisation even with BatchNorm and GELU helping. The model also kept improving later into training and best validation accuracy wasn't hit until epoch 16, whereas the Pancake plateaued around epoch 9. This suggested that the Tower was still extracting useful structure rather than just memorising.

4 Part C: The Champion

4.1 Architecture Design Rationale

Based on the observations from Parts A and B, the Champion model was designed with the following principles:

- Funnel shape where layer widths decrease progressively (784 → 384 → 256 → 128 → 64 → 15) which forced the network to compress representations into increasingly abstract features. This provided a more hierarchical structure than the Pancake while avoiding the slow convergence of the Tower.
- GELU activations for smoother gradient flow.
- BatchNorm after every Linear layer for stabilizing training across all depths.
- Annealed Dropout was applied stronger in early wide layers (0.30) and progressively weaker near the output (0.10) which balanced regularisation with expressive power at each stage.
- Kaiming (He) initialisation for initial weight scales to prevent vanishing gradients at start of training.
- On the fly augmentation applied in the training loop (not the dataset class), keeping the data pipeline unmodified while significantly reducing overfitting.
- **Label smoothing** ($\varepsilon = 0.05$) to prevent model from becoming over confident

Table 5: Champion Model Architecture

Block	Output Dim	Components
Input	784	`x.view(-1, 784)`
Block 1	384	Linear → BN → GELU → Dropout(0.30)
Block 2	256	Linear → BN → GELU → Dropout(0.25)
Block 3	128	Linear → BN → GELU → Dropout(0.20)
Block 4	64	Linear → BN → GELU → Dropout(0.10)
Output	15	Linear (raw logits)

Total Parameters: 443,791
Epochs Trained: 35
Optimiser: AdamW, $\eta = 10^{-3}$, weight decay 2×10^{-4}
Augmentation: On (horizontal flip, random erase, gaussian noise)

4.2 Results

Table 6: Champion Model Performance

Metric	Training (Epoch 35)	Validation (Best)
Accuracy	82.96%	81.10%
Loss	0.7855	0.8312

The train/val gap of only 1.5 percent confirms that the combination of augmentation and dropout effectively eliminated the overfitting observed in the Pancake model. The model converged smoothly with validation accuracy still incrementally improving up to epoch 34-35 before fully plateauing.

5 Part D: Theoretical Analysis

5.1 Width vs. Depth

Table 7: Model Comparison

Model	Parameters	Hidden Layers	Epochs	Best Val Acc
Pancake	1,336,335	2	20	78.23%
Tower	603,151	7	20	78.66%
Champion	443,791	4	35	81.10%

The Universal Approximation Theorem guarantees that a single hidden layer with enough width can approximate any continuous function on a compact domain. Despite this the Pancake, which is basically built around that idea, performs no better than the Tower at roughly 78% validation accuracy while using over twice as many parameters and overfitting far more. There are a few reasons why deep architectures tend to win in practice for image tasks:

1. The theorem guarantees a solution exists but says nothing about how large it needs to be. Deep networks can represent exponentially richer function classes than shallow ones at the same parameter count and the Tower reaches comparable accuracy with 733K fewer parameters.

2. Images have a natural hierarchy as pixels combine into edges, edges into shapes, shapes into objects. A deep network can learn this composition layer by layer. A single wide layer has to capture all of it at once which less efficient and less generalisable.

3. Depth with BatchNorm imposes an inductive bias that favours hierarchical representations, effectively shrinking the hypothesis space and improving generalisation even when the total parameter count is lower.

4. The theorem only says a shallow solution exists and it doesn't say gradient descent will find it. In practice, deeper networks with BatchNorm and GELU converge to better solutions more reliably than very wide shallow ones.

5.2 Confusion Analysis

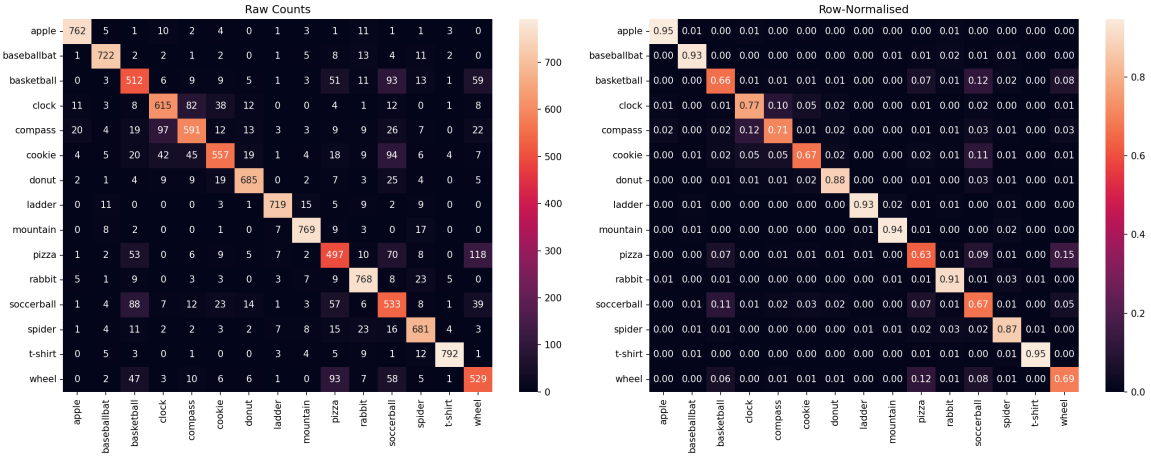

Figure 1: Confusion matrix for the Champion model on the validation set. Left: raw counts. Right: row normalised rates (fraction of each true class predicted as each class).

The two most confused class pairs identified from the confusion matrix are:

1. **Pizza ↔ Wheel:** 15% of pizza samples were misclassified as wheel (118 instances), and 12% of wheel samples were misclassified as pizza (93 instances).
2. **Basketball ↔ Soccerball:** 12% of basketball samples were misclassified as soccerball (93 instances), and 11% of soccerball samples were misclassified as basketball (88 instances).

Both cases are primarily attributable to **aleatoric uncertainty** which is an irreducible ambiguity in the data itself rather than any limitation of the model. Pizza and wheel doodles are both circular with radial lines (pizza slices vs. wheel spokes), and at 28×28 resolution these are often indistinguishable. Basketball and soccerball share the same circular silhouette with internal curve patterns, just representing different stitching designs.

This is different from **epistemic uncertainty**, which arises from model underfitting and could in principle be reduced with more capacity or better training. Here the visual signal is simply absent at this resolution and no amount of additional model capacity will resolve the ambiguity when a pizza and a wheel look identical at 28 pixels wide.

6 Part E: Rationale

6.1 Why This Architecture?

A summary comparison of all three required models (Pancake, Tower, Champion) is provided in Table 7 in Part D. The Champion model's design came directly from what Parts A and B showed:

The Pancake demonstrated that width without depth overfits badly. Two wide layers give plenty of capacity but no hierarchy as the model memorises pixel patterns rather than learning structure.

The Tower showed that depth generalises better with fewer parameters, but seven uniform layers of 256 neurons is somewhat wasteful. The first layer has to go from 784 inputs directly to 256, which is a large compression in one step, and later layers just keep mixing 256-dimensional representations without reducing them.

The funnel design was chosen as the middle ground. A wider first layer (384) captures more variety from the 784 dimensional input. Each subsequent layer compresses progressively ($256 \rightarrow 128 \rightarrow 64$), forcing the network to retain only what is useful for classification. This creates the hierarchical compression that the Tower was trying to achieve, but more efficiently. Four hidden layers were chosen over seven because the Tower showed that the accuracy gains from going very deep flatten out, and deeper networks are slower and harder to train without skip connections.

6.2 How Overfitting Was Combated

Three complementary strategies were used:

- **Annealed Dropout** ($0.30 \rightarrow 0.10$): stronger regularisation in the earlier wide layers where the model has more capacity to overfit, and weaker near the output where we want the model to be more decisive.
- **Weight Decay** (2×10^{-4}) via AdamW: penalises large weights directly, keeping the model from fitting noise in the training set.
- **Data Augmentation** is by far the biggest contributor. Without augmentation the train/val gap was around 17pp (as seen in the Pancake). With horizontal flipping, random patch erasing and gaussian noise, the gap dropped to under 2pp. Augmentation effectively give the model more varied training examples so it can't just memorise fixed pixel patterns.

6.3 Architecture Search

Before arriving at the final Champion, several MLP architectures were tried. Table 8 summarises the results, and Figure 3 shows the full training curves.

Table 8: Architecture Search: MLP Experiments

Model	Params	Epochs	Val Acc	Augmentation
ChampionV2 (wider funnel)	649,935	30	78.50%	Off
ChampionV3 (residual blocks)	973,839	17	78.26%	Off
ChampionV6 (ResFFN + LayerNorm)	2,746,767	22	78.18%	Off
ChampionV7 (deep narrow + SWA)	669,455	32	78.11%	Off
Champion (funnel + aug)	**443,791**	**35**	**81.10%**	**On**

The results reveal a pattern: every architectural variation plateaued at roughly 78% validation accuracy without augmentation, regardless of parameter count. ChampionV6 used 2.7M parameters and still couldn't break past 78.18%. The breakthrough came only when augmentation was added to the original funnel architecture, pushing validation accuracy to 81.10% with the *fewest* parameters of any model tested. This confirms the 78% ceiling was not a capacity problem but an overfitting problem.

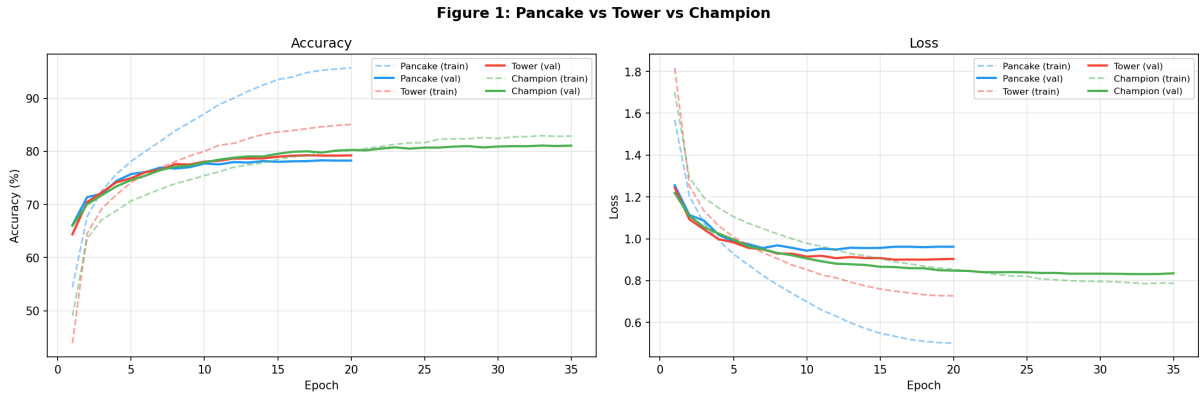

Figure 2: Training and validation accuracy (left) and loss (right). The Pancake's large train/val gap shows severe overfitting. The Tower shows a much smaller gap. The Champion tracks train and val closely throughout due to augmentation.

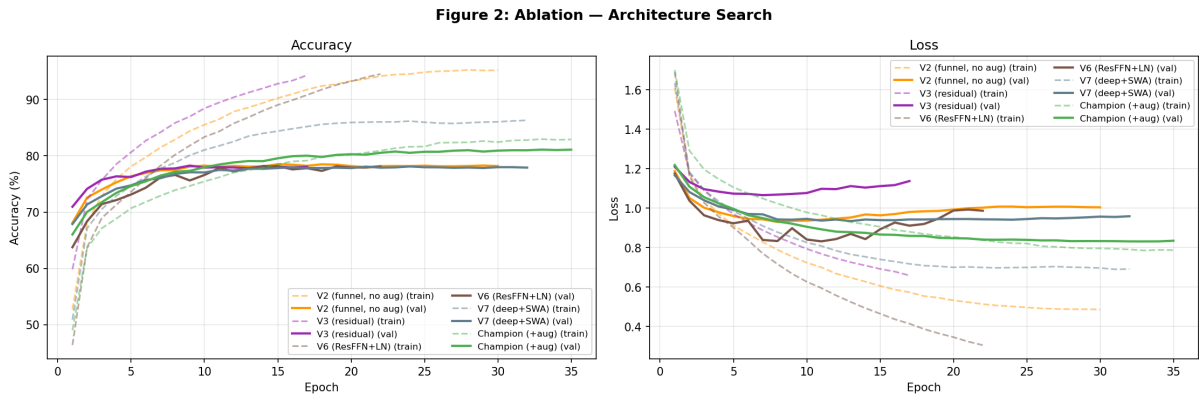

Figure 3: Training curves for all architecture experiments in part C. Every model without augmentation (orange, purple, brown, grey) plateaued near 78% regardless of capacity or architecture choice. The Champion with augmentation (green) is the only model to break past 80%.

The Champion achieves the highest accuracy with the fewest parameters. The extra epochs (35 vs 20) were needed because augmentation slows convergence as the model sees noisier versions of each image so it takes longer to learn features. The payoff is significantly better generalisation, as reflected in the validation accuracy.

7 AI Policy Compliance

The following AI tools were used during this assignment:

- ChatGPT used for assistance . Full conversation log at:
  https://chatgpt.com/share/69a318c1-be38-800d-af67-d7856ab6069e
- VSCode IntelliSense was used for Python syntax autocompletion during development.